



DATABASE ENGINEERING · SQL PROJECT

SQL Project Documentation

Farmer's Market Management System

DATABASE NAME : **farmer**

Made By : **Sambit Kumar Sahu**

1. Project Introduction

A farmer's market is a physical marketplace, typically held on fixed days of the week, where local vendors — farmers, bakers, dairy producers, and craft-food makers — rent booths to sell fresh produce and related goods directly to customers. While such markets appear simple on the surface, the day-to-day operations behind them involve a surprising amount of data: which vendor is assigned to which booth on which date, what products each vendor is selling and at what price, how much inventory a vendor brought to market, and which customers bought what, when, and for how much.

The **Farmer's Market Management System** is a relational database designed to capture exactly this information in a structured, query-able form. Instead of vendors and market organisers relying on paper records or scattered spreadsheets, all data is centralised inside a single relational database named **farmer**. This project documents the design, structure, and query capabilities of that database.

1.1 Business Scenario

Imagine a market organiser who runs a weekly farmer's market across different seasons of the year. Every market day, vendors are assigned to specific booths based on a pricing tier (premium corner booths cost more than standard booths). Each vendor brings a certain quantity of products — categorised into groups such as fresh produce, dairy products, prepared foods, and flowers — and sets an original selling price for each item. Throughout the day, customers walk around the market and make purchases at various vendor booths, and each such purchase is recorded as a transaction with a timestamp, quantity, and price paid.

Over time, this generates a rich transactional history that can answer important business questions: Which products sell best in which season? Which vendors generate the highest revenue? Does rainy weather reduce customer attendance? Which booths are most in demand? The **farmer** database, and the SQL queries built on top of it, are designed to answer exactly these kinds of questions.

1.2 Purpose of the Database

The purpose of the **farmer** database is to provide a single source of truth for all farmer's market operations — vendors, products, booths, customers, and transactions — so that market organisers can:

- Track which vendor occupied which booth on any given market date.
- Maintain an accurate catalogue of products and their categories.
- Record and analyse every customer purchase transaction.
- Monitor vendor inventory levels and pricing across market dates.
- Correlate sales performance with market conditions such as season and weather.
- Generate business reports and insights using structured SQL queries.

2. Project Objectives

The following objectives were defined at the start of this project and guided every stage of design and implementation:

1. To design a normalised relational database schema capable of representing the complete operations of a farmer's market.
2. To correctly identify entities, attributes, and relationships between vendors, products, booths, customers, and market dates.
3. To apply appropriate primary key and foreign key constraints that preserve referential integrity across all eleven tables.
4. To write clean, well-documented `CREATE TABLE` statements that reflect real-world data types and constraints.
5. To demonstrate practical use of core SQL concepts including filtering, sorting, grouping, and aggregation.
6. To demonstrate advanced SQL concepts including joins, subqueries, `CASE` expressions, date functions, and window functions.
7. To extract meaningful, real-world business insights from the data using SQL alone, without any external analytics tool.
8. To document the entire project in a structured, beginner-friendly format suitable for academic submission and future reference.
9. To understand how relational databases model many-to-many and one-to-many relationships using junction tables such as `vendor_booth_assignments`.
10. To build a strong practical foundation in SQL that can be extended to larger, industry-scale database systems in the future.

3. Database Overview

3.1 How the Farmer Database Works

The **farmer** database is composed of **eleven interconnected tables**, each representing a distinct real-world entity or event within the market's operations. At its core, three “master” tables define the main entities of the system:

- **vendor** – the sellers who participate in the market.
- **product** and **product_category** – what is being sold.
- **customer** and **booth** – who buys, and where selling happens.

Around these master tables, several **transactional and relationship tables** record the day-to-day events of the market: which vendor was assigned to which booth (**vendor_booth_assignments**), how much inventory a vendor brought on a given date (**vendor_inventory**), and what customers actually purchased (**customer_purchases** and **customer_purchase_date**). The **market_date_info** table stores metadata about each market day itself — such as the season, weather conditions, and operating hours — while **datetime_demo** is a smaller demonstration table used to practise date and time based SQL functions.

Together, these tables form a connected web of information: a single customer purchase record links directly to a product, a vendor, and a market date, which in turn links to weather and seasonal information for that date. This interconnection is what allows a handful of SQL queries to produce rich, multi-dimensional business insights.

3.2 Importance of Relational Databases

A relational database organises data into multiple related tables rather than one large flat file. This design, first proposed by E. F. Codd in 1970, offers several critical advantages that make it ideal for a system like the Farmer's Market Management System:

- **Reduced Redundancy** – a vendor's name is stored once in the **vendor** table rather than being repeated in every sales record, saving storage and avoiding inconsistency.
- **Data Integrity** – foreign key constraints ensure a purchase record can never reference a product or vendor that does not exist.
- **Flexibility** – new tables (for example, a future **promotions** table) can be added without redesigning the entire system.
- **Efficient Querying** – related data can be combined on demand using **JOIN** operations instead of being permanently duplicated across the database.
- **Scalability** – the same design that works for a small weekly market can scale to a network of markets operating across an entire region.

Key Takeaway

By splitting the Farmer's Market data into eleven purpose-built tables instead of one giant spreadsheet, the database avoids duplicate data, protects data accuracy, and makes complex business questions answerable through simple, reusable SQL queries.

4. Tables Used

The **farmer** database consists of eleven tables. The table below summarises each one along with its purpose within the system.

Table Name	Purpose
booth	Stores information about physical booths available at the market, including their pricing tier and description.
customer	Stores basic demographic information about registered market customers.
customer_purchase_date	Records detailed date/time-stamped customer purchase transactions, including precise market start and end datetimes.
customer_purchases	Stores the core transaction-level record of every item a customer purchased at the market.
datetime_demo	A demonstration table used to practise SQL date and time functions on market scheduling data.
market_date_info	Stores metadata for every market date, including season, weather, and operating hours.
product	Stores the catalogue of products sold at the market and links each product to a category.
product_category	Stores the list of product categories such as Fresh Produce, Dairy, and Flowers.
vendor	Stores information about every vendor who participates in the market.
vendor_booth_assignments	A junction table recording which vendor occupied which booth on which market date.
vendor_inventory	Records how much of each product a vendor brought to the market and at what original price on a given date.

5. Entity Relationship Description

5.1 Overview of Relationships

The **farmer** database follows a typical star-like transactional design, where a small number of master (reference) tables are connected to larger transactional tables through foreign keys. The relationships can be grouped into the following categories:

5.1.1 One-to-Many Relationships

- **product_category** → **product**: one category can contain many products, but each product belongs to exactly one category.
- **vendor** → **vendor_inventory**: one vendor can supply many inventory records across different market dates.
- **product** → **vendor_inventory**: one product can appear in many vendors' inventories.
- **customer** → **customer_purchases**: one customer can make many purchases over time.
- **market_date_info** → **vendor_booth_assignments**: one market date can have many booth assignments.
- **booth** → **vendor_booth_assignments**: one booth can be assigned on many different market dates.

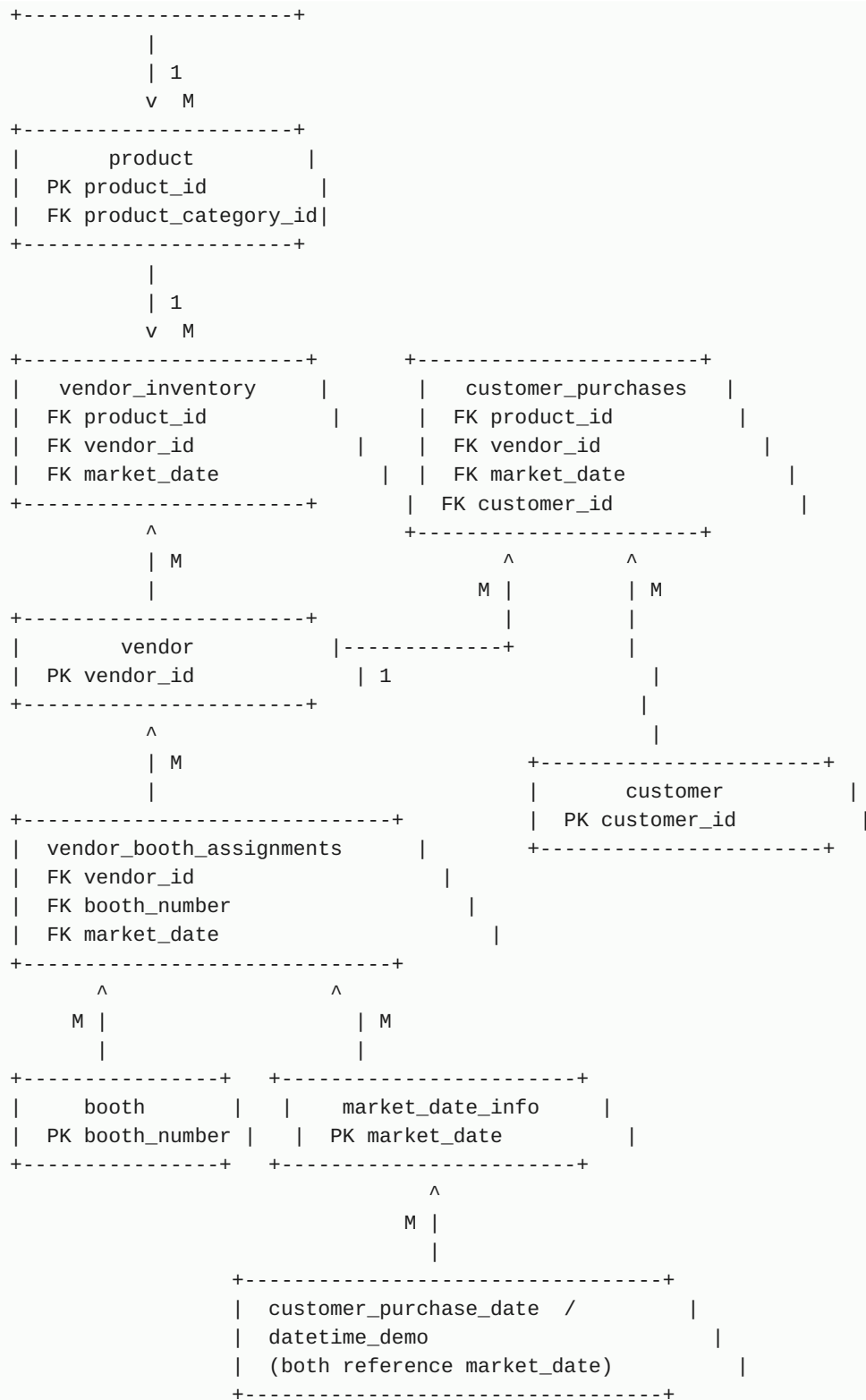
5.1.2 Many-to-Many Relationships (resolved through junction tables)

- **vendor** ↔ **booth**: a vendor can occupy different booths on different dates, and a booth can be occupied by different vendors on different dates. This many-to-many relationship is resolved through the junction table **vendor_booth_assignments**, which combines **vendor_id**, **booth_number**, and **market_date**.
- **vendor** ↔ **product**: a vendor can sell many products, and a product can be sold by many vendors. This relationship is resolved through **vendor_inventory** and **customer_purchases**.
- **customer** ↔ **product**: a customer can purchase many different products from many different vendors. This is resolved through the **customer_purchases** transaction table.

5.2 Text-Based Entity Relationship (ER) Diagram

The diagram below represents the relationships among all eleven tables in text form. Arrows indicate the direction of the foreign key reference (child table points to parent table).

```
+-----+
| product_category |
| PK product_category_id
```



Legend: 1 = "one" side, M = "many" side.
Arrows point from the many (child / foreign key) side
toward the one (parent / primary key) side.

5.3 Relationship Explanations in Simple Language

- **Vendor and Booth:** Every market day, a vendor is assigned to one or more booths. This is not a permanent assignment — the same vendor might sit at a different booth next week — so the relationship is tracked separately in `vendor_booth_assignments` rather than being stored directly inside the `vendor` or `booth` tables.
- **Vendor and Product:** A vendor does not "own" a product permanently; instead, the `vendor_inventory` table tracks which products a vendor brought to market on a specific date, and at what price.
- **Customer and Purchase:** A customer's shopping activity is captured entirely inside `customer_purchases`, which links back to the customer, the vendor they bought from, the product they bought, and the date of purchase.
- **Market Date and Weather:** The `market_date_info` table acts as a calendar of market days, storing contextual details like season and weather that can later be joined against sales data to analyse trends.
- **Product and Category:** Every product belongs to exactly one category (such as Dairy or Fresh Produce), allowing sales to be analysed at both the individual product level and the broader category level.

6. Constraints

Constraints are rules enforced on data columns to maintain accuracy and reliability of the data inside a table. The **farmer** database makes use of the following constraint types.

6.1 Primary Key

A **Primary Key** uniquely identifies each record in a table. It cannot contain **NULL** values and must be unique across all rows. In this project, `vendor_id` in `vendor`, `customer_id` in `customer`, and `product_id` in `product` are all primary keys. Composite primary keys (made of more than one column) are used in transactional tables such as `vendor_inventory`, where the combination of `market_date`, `vendor_id`, and `product_id` uniquely identifies each inventory record.

6.2 Foreign Key

A **Foreign Key** creates a link between two tables by referencing the primary key of another table. It ensures referential integrity — for example, `customer_purchases.customer_id` is a foreign key referencing `customer.customer_id`, which guarantees that a purchase record can never reference a customer that does not exist in the `customer` table.

6.3 Unique

The **UNIQUE** constraint ensures that all values in a column are distinct from one another. In this project it can be applied to `product_category_name` in `product_category`, so that the same category (for example, "Dairy Products") cannot accidentally be entered twice under two different IDs.

6.4 Not Null

The **NOT NULL** constraint ensures that a column must always contain a value. Columns such as `vendor_name`, `product_name`, and `market_date` are marked **NOT NULL** because a vendor without a name, a product without a name, or a transaction without a date would make no logical sense.

6.5 Default

The **DEFAULT** constraint automatically assigns a predefined value to a column when no value is explicitly provided. In this project, the flag columns `market_rain_flag` and `market_snow_flag` in `market_date_info` default to 0 (meaning "no rain" or "no snow") unless stated otherwise.

6.6 Check

The **CHECK** constraint restricts the values that can be stored in a column based on a logical condition. In this project, a **CHECK** constraint can ensure that **quantity** in **customer_purchases** and **vendor_inventory** is always greater than zero, and that **booth_price_level** only accepts positive integer values.

6.7 Summary Table of Constraint Usage

Constraint	Applied On	Purpose
PRIMARY KEY	vendor_id, customer_id, product_id, booth_number, product_category_id	Uniquely identify each record
FOREIGN KEY	product_category_id, vendor_id, product_id, customer_id, booth_number, market_date	Maintain referential integrity between tables
UNIQUE	product_category_name	Prevent duplicate category names
NOT NULL	vendor_name, product_name, market_date, customer_last_name	Ensure mandatory fields always contain data
DEFAULT	market_rain_flag, market_snow_flag	Auto-fill weather flags when not specified
CHECK	quantity, booth_price_level, original_price	Restrict values to logically valid ranges

7. Database Creation

Before any table can be created, the database itself must be created and selected for use. The following statements accomplish this in MySQL-style SQL syntax:

```
CREATE DATABASE farmer;  
  
USE farmer;
```

The `CREATE DATABASE` statement instructs the database server to allocate a new, empty database named **farmer**. This database acts as a container that will hold all eleven tables described in this document, along with their relationships, indexes, and constraints. The `USE` statement then tells the database server that all subsequent SQL statements — such as `CREATE TABLE`, `INSERT`, and `SELECT` — should be executed within the context of the **farmer** database, rather than any other database that might exist on the same server.

Good Practice

It is good practice to always confirm the active database using `SELECT DATABASE ( ) ;` before running structural changes, especially on a shared development server where multiple project databases may exist side by side.

8. Table Description

This section describes each of the eleven tables in detail, including its purpose, column definitions, primary key, foreign keys, and the SQL statement used to create it.

8.1 booth

Purpose: Stores the master list of physical booths available at the market, along with their pricing tier and description.

Column	Data Type	Description
booth_number	INT	Unique number identifying a booth. Serves as the Primary Key.
booth_price_level	VARCHAR(20)	Pricing tier of the booth (e.g. Standard, Premium, Corner).
booth_description	VARCHAR(255)	Short textual description of the booth's location or features.
booth_type	VARCHAR(50)	Type of booth, such as Indoor, Outdoor, or Covered Stall.

Primary Key: booth_number | **Foreign Keys:** None

```
CREATE TABLE booth (  
    booth_number      INT PRIMARY KEY,  
    booth_price_level VARCHAR(20) NOT NULL,  
    booth_description VARCHAR(255),  
    booth_type        VARCHAR(50) NOT NULL  
);
```

8.2 customer

Purpose: Stores basic demographic details of every registered market customer.

Column	Data Type	Description
customer_id	INT	Unique identifier for a customer. Serves as the Primary Key.
customer_first_name	VARCHAR(50)	Customer's first name.
customer_last_name	VARCHAR(50)	Customer's last name.
customer_zip	VARCHAR(10)	Postal / ZIP code of the customer's residence, useful for geographic analysis.

Primary Key: customer_id | **Foreign Keys:** None

```
CREATE TABLE customer (
  customer_id          INT PRIMARY KEY,
  customer_first_name  VARCHAR(50) NOT NULL,
  customer_last_name   VARCHAR(50) NOT NULL,
  customer_zip         VARCHAR(10)
);
```

8.3 product_category

Purpose: Stores the master list of product categories used to classify products sold at the market.

Column	Data Type	Description
product_category_id	INT	Unique identifier for a product category. Serves as the Primary Key.
product_category_name	VARCHAR(50)	Name of the category, e.g. Fresh Produce, Dairy, Flowers, Prepared Food.

Primary Key: product_category_id | **Foreign Keys:** None

```
CREATE TABLE product_category (
  product_category_id  INT PRIMARY KEY,
  product_category_name VARCHAR(50) NOT NULL UNIQUE
);
```

8.4 product

Purpose: Stores the catalogue of individual products that vendors may bring to the market.

Column	Data Type	Description
product_id	INT	Unique identifier for a product. Serves as the Primary Key.
product_name	VARCHAR(100)	Name of the product, e.g. "Roma Tomatoes" or "Goat Cheese".
product_size	VARCHAR(30)	Descriptive size of the product, e.g. "500g", "1 Dozen".
product_category_id	INT	References the category this product belongs to. Foreign Key.
product_qty_type	VARCHAR(20)	Unit of measure used when selling the product, e.g. lb, unit, bunch.

Primary Key: product_id | **Foreign Keys:** product_category_id → product_category(product_category_id)

```
CREATE TABLE product (
  product_id          INT PRIMARY KEY,
  product_name        VARCHAR(100) NOT NULL,
  product_size        VARCHAR(30),
  product_category_id INT,
  product_qty_type    VARCHAR(20),
  CONSTRAINT fk_product_category
    FOREIGN KEY (product_category_id)
    REFERENCES product_category (product_category_id)
);
```

8.5 vendor

Purpose: Stores information about every vendor registered to sell at the market.

Column	Data Type	Description
vendor_id	INT	Unique identifier for a vendor. Serves as the Primary Key.
vendor_name	VARCHAR(100)	Business or stall name used by the vendor.
vendor_type	VARCHAR(50)	Category of vendor, e.g. Fruit & Vegetables, Bakery, Dairy.
vendor_owner_first_name	VARCHAR(50)	First name of the vendor's owner.
vendor_owner_last_name	VARCHAR(50)	Last name of the vendor's owner.

Primary Key: vendor_id | **Foreign Keys:** None

```
CREATE TABLE vendor (
  vendor_id          INT PRIMARY KEY,
  vendor_name        VARCHAR(100) NOT NULL,
  vendor_type        VARCHAR(50),
  vendor_owner_first_name VARCHAR(50),
  vendor_owner_last_name VARCHAR(50)
);
```

8.6 market_date_info

Purpose: Stores calendar and contextual metadata for every market date, including weather and seasonal information.

Column	Data Type	Description
market_date	DATE	The calendar date of the market. Serves as the Primary Key.
market_day	VARCHAR(15)	Day of the week, e.g. Saturday.

market_week	INT	Week number within the market year.
market_year	INT	Calendar year of the market date.
market_start_time	TIME	Scheduled opening time of the market.
market_end_time	TIME	Scheduled closing time of the market.
special_notes	VARCHAR(255)	Free-text notes, e.g. "Holiday Market" or "Vendor Fair".
market_season	VARCHAR(20)	Season during which the market date falls, e.g. Spring, Summer.
market_min_temp	DECIMAL(4,1)	Minimum recorded temperature (°C) on the market date.
market_max_temp	DECIMAL(4,1)	Maximum recorded temperature (°C) on the market date.
market_rain_flag	TINYINT(1)	1 if it rained on the market date, otherwise 0. Defaults to 0.
market_snow_flag	TINYINT(1)	1 if it snowed on the market date, otherwise 0. Defaults to 0.

Primary Key: market_date | **Foreign Keys:** None

```
CREATE TABLE market_date_info (
  market_date      DATE PRIMARY KEY,
  market_day       VARCHAR(15),
  market_week      INT,
  market_year      INT,
  market_start_time TIME,
  market_end_time  TIME,
  special_notes     VARCHAR(255),
  market_season    VARCHAR(20),
  market_min_temp  DECIMAL(4,1),
  market_max_temp  DECIMAL(4,1),
  market_rain_flag TINYINT(1) DEFAULT 0,
  market_snow_flag TINYINT(1) DEFAULT 0
);
```

8.7 vendor_booth_assignments

Purpose: A junction table that records which vendor occupied which booth on a specific market date, resolving the many-to-many relationship between vendors and booths.

Column	Data Type	Description
vendor_id	INT	References the assigned vendor. Foreign Key, part of composite Primary Key.
booth_number	INT	References the assigned booth. Foreign Key, part of composite Primary Key.

market_date	DATE	Date of the assignment. Foreign Key, part of composite Primary Key.
-------------	------	---

Primary Key: Composite (vendor_id, booth_number, market_date) | **Foreign Keys:** vendor_id → vendor(vendor_id), booth_number → booth(booth_number), market_date → market_date_info(market_date)

```
CREATE TABLE vendor_booth_assignments (
  vendor_id      INT NOT NULL,
  booth_number   INT NOT NULL,
  market_date    DATE NOT NULL,
  PRIMARY KEY (vendor_id, booth_number, market_date),
  CONSTRAINT fk_vba_vendor
    FOREIGN KEY (vendor_id) REFERENCES vendor (vendor_id),
  CONSTRAINT fk_vba_booth
    FOREIGN KEY (booth_number) REFERENCES booth (booth_number),
  CONSTRAINT fk_vba_market_date
    FOREIGN KEY (market_date) REFERENCES market_date_info (market_date)
);
```

8.8 vendor_inventory

Purpose: Records the quantity and original price of each product a vendor brought to the market on a given date.

Column	Data Type	Description
market_date	DATE	Date the inventory applies to. Foreign Key, part of composite Primary Key.
vendor_id	INT	Vendor who supplied the inventory. Foreign Key, part of composite Primary Key.
product_id	INT	Product being stocked. Foreign Key, part of composite Primary Key.
quantity	DECIMAL(8,2)	Quantity of the product brought to market.
original_price	DECIMAL(8,2)	Vendor's listed original selling price per unit.

Primary Key: Composite (market_date, vendor_id, product_id) | **Foreign Keys:** market_date → market_date_info(market_date), vendor_id → vendor(vendor_id), product_id → product(product_id)

```
CREATE TABLE vendor_inventory (
  market_date    DATE NOT NULL,
  vendor_id      INT NOT NULL,
```

```

product_id      INT          NOT NULL,
quantity        DECIMAL(8,2) CHECK (quantity >= 0),
original_price  DECIMAL(8,2) CHECK (original_price >= 0),
PRIMARY KEY (market_date, vendor_id, product_id),
CONSTRAINT fk_vi_market_date
    FOREIGN KEY (market_date) REFERENCES market_date_info (market_date),
CONSTRAINT fk_vi_vendor
    FOREIGN KEY (vendor_id) REFERENCES vendor (vendor_id),
CONSTRAINT fk_vi_product
    FOREIGN KEY (product_id) REFERENCES product (product_id)
);

```

8.9 customer_purchases

Purpose: The core transactional table that records every individual item a customer purchased at the market.

Column	Data Type	Description
product_id	INT	Product purchased. Foreign Key, part of composite Primary Key.
vendor_id	INT	Vendor the purchase was made from. Foreign Key, part of composite Primary Key.
market_date	DATE	Date of the purchase. Foreign Key, part of composite Primary Key.
customer_id	INT	Customer who made the purchase. Foreign Key, part of composite Primary Key.
quantity	DECIMAL(8,2)	Quantity of the product purchased.
cost_to_customer_per_qty	DECIMAL(8,2)	Actual price per unit paid by the customer.
transaction_time	TIME	Time of day the transaction occurred. Part of composite Primary Key.

Primary Key: Composite (market_date, customer_id, vendor_id, product_id, transaction_time) | **Foreign Keys:** product_id → product(product_id), vendor_id → vendor(vendor_id), market_date → market_date_info(market_date), customer_id → customer(customer_id)

```

CREATE TABLE customer_purchases (
    product_id      INT          NOT NULL,
    vendor_id        INT          NOT NULL,
    market_date      DATE         NOT NULL,
    customer_id      INT          NOT NULL,
    quantity         DECIMAL(8,2) CHECK (quantity > 0),

```

```

    cost_to_customer_per_qty    DECIMAL(8,2)  CHECK (cost_to_customer_per_qty
    >= 0),
    transaction_time            TIME           NOT NULL,
    PRIMARY KEY (market_date, customer_id, vendor_id, product_id,
    transaction_time),
    CONSTRAINT fk_cp_product
        FOREIGN KEY (product_id) REFERENCES product (product_id),
    CONSTRAINT fk_cp_vendor
        FOREIGN KEY (vendor_id) REFERENCES vendor (vendor_id),
    CONSTRAINT fk_cp_market_date
        FOREIGN KEY (market_date) REFERENCES market_date_info (market_date),
    CONSTRAINT fk_cp_customer
        FOREIGN KEY (customer_id) REFERENCES customer (customer_id)
);

```

8.10 customer_purchase_date

Purpose: An extended transaction-log table that captures fine-grained date and time detail for each customer purchase, including both the scheduled market window and the precise transaction timestamp. This table is primarily used for teaching and practising SQL date/time functions on real transaction-shaped data.

Column	Data Type	Description
market_date	DATE	Calendar date of the market session.
market_start_time	TIME	Scheduled opening time of that market session.
market_end_time	TIME	Scheduled closing time of that market session.
market_start_datetime	DATETIME	Combined date and start time of the market session.
market_end_datetime	DATETIME	Combined date and end time of the market session.
market_date_transaction	DATE	The date on which the customer transaction actually occurred.
transaction_time	TIME	Exact time the transaction occurred.
product_id	INT	Product purchased. Foreign Key.
vendor_id	INT	Vendor the purchase was made from. Foreign Key.
customer_id	INT	Customer who made the purchase. Foreign Key.
quantity	DECIMAL(8,2)	Quantity purchased.
cost_to_customer_per_qty	DECIMAL(8,2)	Price per unit paid by the customer.

Primary Key: Composite (market_date_transaction, customer_id, vendor_id, product_id, transaction_time) | **Foreign Keys:** product_id → product(product_id),

vendor_id → vendor(vendor_id), customer_id → customer(customer_id),
market_date → market_date_info(market_date)

```
CREATE TABLE customer_purchase_date (  
    market_date          DATE          NOT NULL,  
    market_start_time    TIME,  
    market_end_time      TIME,  
    market_start_datetime DATETIME,  
    market_end_datetime  DATETIME,  
    market_date_transaction DATE      NOT NULL,  
    transaction_time      TIME      NOT NULL,  
    product_id           INT         NOT NULL,  
    vendor_id            INT         NOT NULL,  
    customer_id          INT         NOT NULL,  
    quantity             DECIMAL(8,2),  
    cost_to_customer_per_qty DECIMAL(8,2),  
    PRIMARY KEY (market_date_transaction, customer_id, vendor_id, product_id,  
transaction_time),  
    CONSTRAINT fk_cpd_product  
        FOREIGN KEY (product_id) REFERENCES product (product_id),  
    CONSTRAINT fk_cpd_vendor  
        FOREIGN KEY (vendor_id) REFERENCES vendor (vendor_id),  
    CONSTRAINT fk_cpd_customer  
        FOREIGN KEY (customer_id) REFERENCES customer (customer_id),  
    CONSTRAINT fk_cpd_market_date  
        FOREIGN KEY (market_date) REFERENCES market_date_info (market_date)  
);
```

8.11 datetime_demo

Purpose: A small, standalone demonstration table used purely for practising SQL date and time functions (such as `EXTRACT()`, `DATEDIFF()`, and `TIMESTAMPDIFF()`) without the complexity of the full transactional schema.

Column	Data Type	Description
market_date	DATE	Calendar date of a sample market session. Serves as the Primary Key.
market_start_time	TIME	Sample opening time.
market_end_time	TIME	Sample closing time.
market_start_datetime	DATETIME	Combined date and start time.
market_end_datetime	DATETIME	Combined date and end time.

Primary Key: market_date | **Foreign Keys:** None (standalone practice table)

```
CREATE TABLE datetime_demo (  
    market_date          DATE PRIMARY KEY,  
    market_start_time    TIME,  
    market_end_time      TIME,  
    market_start_datetime DATETIME,  
    market_end_datetime  DATETIME  
);
```

9. Data Insertion

Once the eleven tables of the **farmer** database were created with their appropriate constraints, the next stage of the project involved populating them with realistic sample data. Data was inserted in a carefully chosen order that respects the referential integrity of the schema — independent "master" tables such as `product_category`, `vendor`, `customer`, and `booth` were populated first, since they have no dependency on any other table. Only after these master tables contained valid records were the dependent tables populated: `product` (which depends on `product_category`), followed by `market_date_info`, and finally the transactional and relationship tables — `vendor_booth_assignments`, `vendor_inventory`, `customer_purchases`, and `customer_purchase_date` — which depend on nearly every other table in the schema. Populating records in this order ensured that no foreign key constraint was ever violated during insertion, and that every purchase, inventory, and booth assignment record could be traced back to a valid vendor, product, customer, booth, and market date. Sample data was chosen to span multiple seasons and market dates so that the SQL queries in the following section could demonstrate meaningful, real-world patterns such as seasonal sales trends and weather-linked attendance changes.

10. SQL Queries

This section presents fifteen SQL queries written against the **farmer** database. The queries are arranged in order of increasing difficulty and, together, cover nearly every major concept in the SQL language — from basic filtering to window functions.

Q1 – Simple SELECT

Question: Display the ID, name, and type of every vendor registered in the market.

SQL QUERY

```
SELECT vendor_id, vendor_name, vendor_type
FROM vendor;
```

EXPLANATION

This is the most basic form of data retrieval. The **SELECT** clause specifies exactly which columns to return, and **FROM vendor** tells the database which table to read from. No filtering or sorting is applied, so every row in the **vendor** table is returned.

EXPECTED RESULT

```
vendor_id | vendor_name | vendor_type | -----+-----
+----- 1 | Green Valley Farms | Fruit & Vegetables 2 | Sunrise
Dairy Co. | Dairy 3 | Wildflower Blooms | Flowers ...
```

Q2 – WHERE Clause

Question: List all products that belong to the ‘Fresh Produce’ category (product_category_id = 1).

SQL QUERY

```
SELECT product_id, product_name, product_size
FROM product
WHERE product_category_id = 1;
```

EXPLANATION

The **WHERE** clause filters rows before they are returned, keeping only the products whose **product_category_id** equals 1. This demonstrates how conditional filtering narrows a large result set down to only the relevant records.

EXPECTED RESULT

product_id	product_name	product_size
101	Roma Tomatoes	500g
102	Sweet Corn	6 pack
103	Baby Spinach	250g

Q3 – ORDER BY

Question: List all vendors sorted alphabetically by vendor name.

SQL QUERY

```
SELECT vendor_id, vendor_name
FROM vendor
ORDER BY vendor_name ASC;
```

EXPLANATION

The **ORDER BY** clause sorts the final result set. Here, **ASC** arranges vendor names from A to Z. Sorting does not change the underlying table — it only changes the order in which rows are displayed to the user.

EXPECTED RESULT

vendor_id	vendor_name
1	Green Valley Farms
2	Sunrise Dairy Co.
3	Wildflower Blooms

Q4 – DISTINCT

Question: Find the distinct list of vendor types participating in the market.

SQL QUERY

```
SELECT DISTINCT vendor_type
FROM vendor;
```

EXPLANATION

The **DISTINCT** keyword removes duplicate values from the result set. Even if ten vendors share the vendor type “Dairy,” **DISTINCT** ensures that “Dairy” is listed only once in the output, giving a clean summary of all unique categories of vendors.

EXPECTED RESULT

```
vendor_type ----- Fruit & Vegetables Dairy Flowers Bakery
```

Q5 – Aggregate Function

Question: Calculate the total revenue generated across all customer purchases recorded in the database.

SQL QUERY

```
SELECT SUM(quantity * cost_to_customer_per_qty) AS total_revenue
FROM customer_purchases;
```

EXPLANATION

Aggregate functions such as **SUM()** collapse many rows into a single summarised value. Here, each purchase’s revenue is calculated as **quantity × cost_to_customer_per_qty**, and **SUM()** adds these amounts across every row in **customer_purchases** to give the overall total revenue of the market.

EXPECTED RESULT

```
total_revenue ----- 184362.50
```

Q6 – GROUP BY

Question: Find the total quantity sold for each product.

SQL QUERY

```
SELECT product_id, SUM(quantity) AS total_qty_sold
FROM customer_purchases
GROUP BY product_id;
```

EXPLANATION

GROUP BY organises rows that share the same `product_id` into a single group, and `SUM(quantity)` is then calculated separately within each group. This turns a long list of individual transactions into a concise, per-product sales summary.

EXPECTED RESULT

```
product_id | total_qty_sold -----+----- 101 | 452.00 102 |
318.50 103 | 276.00
```

Q7 – HAVING

Question: Identify vendors whose total revenue exceeds 5,000.

SQL QUERY

```
SELECT vendor_id,
       SUM(quantity * cost_to_customer_per_qty) AS total_revenue
FROM customer_purchases
GROUP BY vendor_id
HAVING SUM(quantity * cost_to_customer_per_qty) > 5000;
```

EXPLANATION

While **WHERE** filters individual rows before grouping, **HAVING** filters groups *after* aggregation. This query first calculates each vendor's total revenue and then keeps only the vendors whose grouped total exceeds 5,000 — something a plain **WHERE** clause cannot do, since `total_revenue` does not exist until after grouping.

EXPECTED RESULT

```
vendor_id | total_revenue -----+----- 1 | 18420.00 4 | 9875.50
```

Q8 – INNER JOIN

Question: Display each purchase transaction along with the name of the product purchased.

SQL QUERY

```
SELECT cp.market_date, p.product_name, cp.quantity,  
       cp.cost_to_customer_per_qty  
FROM customer_purchases cp  
INNER JOIN product p  
      ON cp.product_id = p.product_id;
```

EXPLANATION

An **INNER JOIN** combines rows from two tables based on a matching condition — here, matching `product_id` in `customer_purchases` with `product_id` in `product`. Only rows with a match in both tables are returned, which enriches each raw transaction with a readable product name.

EXPECTED RESULT

market_date	product_name	quantity	cost_to_customer_per_qty
2025-06-07	Roma		
2025-06-07	Sweet Corn		

Q9 – LEFT JOIN

Question: Identify vendors who have never been assigned to a booth.

SQL QUERY

```
SELECT v.vendor_id, v.vendor_name, vba.booth_number  
FROM vendor v  
LEFT JOIN vendor_booth_assignments vba  
      ON v.vendor_id = vba.vendor_id  
WHERE vba.booth_number IS NULL;
```

EXPLANATION

A **LEFT JOIN** returns every row from the left table (`vendor`), along with matching rows from the right table (`vendor_booth_assignments`) where available. When a vendor has no matching booth assignment, the joined columns come back as **NULL**. Filtering for `vba.booth_number IS NULL` therefore isolates vendors who have not yet been assigned a booth.

EXPECTED RESULT

vendor_id	vendor_name	booth_number
7	Maple Ridge Honey	NULL

Q10 – Multiple Table JOIN

Question: Produce a full purchase report showing the customer's name, the vendor's name, the product purchased, and the purchase details.

SQL QUERY

```
SELECT c.customer_first_name, c.customer_last_name,  
       v.vendor_name, p.product_name,  
       cp.market_date, cp.quantity, cp.cost_to_customer_per_qty  
FROM customer_purchases cp  
JOIN customer c ON cp.customer_id = c.customer_id  
JOIN vendor v   ON cp.vendor_id   = v.vendor_id  
JOIN product p  ON cp.product_id  = p.product_id;
```

EXPLANATION

This query chains three separate **JOIN** operations onto the central **customer_purchases** transaction table, pulling in readable details from **customer**, **vendor**, and **product** in a single pass. This mirrors how a real sales report would be generated for market management staff.

EXPECTED RESULT

customer_first_name	vendor_name	product_name	market_date	quantity	cost_to_customer_per_qty
Green Valley Farms	Roma Tomatoes		2025-06-07	3.00	2.50

Q11 – Subquery

Question: Find customers whose total spending is greater than the average total spending across all customers.

SQL QUERY

```
SELECT customer_id,  
       SUM(quantity * cost_to_customer_per_qty) AS total_spent  
FROM customer_purchases  
GROUP BY customer_id  
HAVING SUM(quantity * cost_to_customer_per_qty) >  
       (SELECT AVG(customer_total)  
        FROM (  
          SELECT SUM(quantity * cost_to_customer_per_qty) AS customer_total  
          FROM customer_purchases  
          GROUP BY customer_id  
        ) AS customer_totals);
```

EXPLANATION

The inner query calculates the total amount spent by each customer and wraps it in a derived table. The outer `AVG()` subquery then calculates the average of those per-customer totals. Finally, the main query keeps only customers whose own total spending exceeds this database-wide average — a classic example of a subquery being used inside a **HAVING** clause.

EXPECTED RESULT

customer_id	total_spent
12	842.50
27	915.00

Q12 – CASE Statement

Question: Classify each vendor into a performance tier based on their total revenue.

SQL QUERY

```
SELECT vendor_id,  
       SUM(quantity * cost_to_customer_per_qty) AS revenue,  
       CASE  
         WHEN SUM(quantity * cost_to_customer_per_qty) >= 10000 THEN  
           'High Performer'  
         WHEN SUM(quantity * cost_to_customer_per_qty) >= 5000 THEN  
           'Medium Performer'  
         ELSE 'Low Performer'  
       END AS performance_tier  
FROM customer_purchases  
GROUP BY vendor_id;
```

EXPLANATION

The **CASE** expression works like an if-else statement inside SQL. Each vendor's total revenue is evaluated against a series of conditions, and the first matching condition determines the label placed in the `performance_tier` column. This is a common technique for turning raw numbers into readable business categories.

EXPECTED RESULT

```
vendor_id | revenue | performance_tier -----+-----  
+----- 1 | 18420.00 | High Performer 4 | 6875.50 | Medium  
Performer 7 | 2140.00 | Low Performer
```

Q13 – Date Functions

Question: Determine total revenue generated in each calendar month.

SQL QUERY

```
SELECT MONTH(market_date) AS sales_month,  
       SUM(quantity * cost_to_customer_per_qty) AS monthly_revenue  
FROM customer_purchases  
GROUP BY MONTH(market_date)  
ORDER BY sales_month;
```

EXPLANATION

The built-in `MONTH()` function extracts just the month number from a `DATE` value. Grouping by this extracted month allows revenue to be aggregated across an entire month regardless of the year or exact day, which is useful for spotting seasonal sales patterns.

EXPECTED RESULT

```
sales_month | monthly_revenue -----+----- 5 | 21500.00 6 |  
28430.50 7 | 31220.00
```

Q14 – Ranking using Window Function

Question: Rank all vendors by their total revenue, from highest to lowest.

SQL QUERY

```
SELECT vendor_id,  
       SUM(quantity * cost_to_customer_per_qty) AS revenue,  
       RANK() OVER (ORDER BY SUM(quantity * cost_to_customer_per_qty) DESC)  
AS revenue_rank  
FROM customer_purchases  
GROUP BY vendor_id;
```

EXPLANATION

The `RANK()` window function assigns a rank to each row based on the ordering specified inside `OVER()`, without collapsing the result set the way `GROUP BY` alone would. Unlike a subquery-based approach, window functions can compute this ranking in a single, efficient pass, and automatically handle tied revenue values by assigning them the same rank.

EXPECTED RESULT

```
vendor_id | revenue | revenue_rank -----+-----+----- 1 |  
18420.00 | 1 4 | 9875.50 | 2 7 | 2140.00 | 3
```

Q15 – Business Insight Query

Question: For each product category, identify the vendor who generated the highest revenue within that category.

SQL QUERY

```
SELECT product_category_name, vendor_name, category_revenue
FROM (
    SELECT pc.product_category_name,
           v.vendor_name,
           SUM(cp.quantity * cp.cost_to_customer_per_qty) AS
category_revenue,
           RANK() OVER (
               PARTITION BY pc.product_category_name
               ORDER BY SUM(cp.quantity * cp.cost_to_customer_per_qty) DESC
           ) AS category_rank
    FROM customer_purchases cp
    JOIN product p          ON cp.product_id = p.product_id
    JOIN product_category pc ON p.product_category_id =
pc.product_category_id
    JOIN vendor v          ON cp.vendor_id = v.vendor_id
    GROUP BY pc.product_category_name, v.vendor_name
) ranked_vendors
WHERE category_rank = 1;
```

EXPLANATION

This final query brings together nearly every concept covered in this section. Three tables are joined to connect a raw purchase to its product category and vendor. The results are then grouped by category and vendor to calculate total revenue, and the `RANK()` window function is partitioned *by category* so that ranking restarts for every new category. Wrapping this in an outer query and filtering for `category_rank = 1` isolates exactly one top-performing vendor per category — a genuinely useful piece of business intelligence produced entirely with SQL.

EXPECTED RESULT

```
product_category_name | vendor_name | category_revenue -----
+-----+-----+-----+-----+-----+-----+
18420.00 Dairy Products | Sunrise Dairy Co. | 12300.00 Flowers | Wildflower
Blooms | 5410.00
```


11. Business Insights

Beyond individual queries, the true value of the **farmer** database lies in the business insights it can generate for market organisers and vendors. The following insights can be derived from the data using the SQL techniques demonstrated in this report.

Insight	How SQL Derives It
Top Selling Products	Grouping <code>customer_purchases</code> by <code>product_id</code> and ordering by total quantity sold reveals which products customers buy the most.
Highest Earning Vendor	Grouping purchases by <code>vendor_id</code> , summing revenue, and ranking with <code>RANK()</code> identifies the single top-performing vendor.
Most Active Customer	Counting transactions per <code>customer_id</code> and sorting in descending order identifies the most frequent market shopper.
Products With Low Inventory	Comparing <code>vendor_inventory.quantity</code> against a defined minimum threshold flags products at risk of running out.
Season-Wise Sales	Joining <code>customer_purchases</code> with <code>market_date_info</code> and grouping by <code>market_season</code> reveals which season generates the most revenue.
Most Occupied Booths	Grouping <code>vendor_booth_assignments</code> by <code>booth_number</code> and counting rows identifies the most consistently used booths.
Vendor Performance Comparison	Using CASE expressions on grouped vendor revenue classifies vendors into performance tiers for easy comparison.
Market Attendance Trends	Counting distinct <code>customer_id</code> values per <code>market_date</code> shows whether attendance is growing or shrinking over time.
Weather Impact on Sales	Joining sales data with <code>market_rain_flag</code> and <code>market_snow_flag</code> from <code>market_date_info</code> reveals how weather conditions affect total revenue.
Category-Level Demand	Joining <code>product</code> , <code>product_category</code> , and <code>customer_purchases</code> shows which product categories (e.g. Dairy vs. Fresh Produce) drive the most overall demand.

Why This Matters

None of these insights require any tool beyond SQL. Every one of them can be generated directly from the **farmer** database using the same `JOIN`, `GROUP BY`, `CASE`, and window function techniques demonstrated in Section 11, reinforcing how a well-designed relational schema turns raw transactional data into actionable business intelligence.

12. Learning Outcomes

By completing this project, the following practical learning outcomes were achieved:

1. Developed a clear understanding of relational database design principles, including entities, attributes, and relationships.
2. Gained hands-on experience designing a normalised schema comprising eleven interrelated tables.
3. Learned to correctly apply **PRIMARY KEY**, **FOREIGN KEY**, **UNIQUE**, **NOT NULL**, **DEFAULT**, and **CHECK** constraints to enforce data integrity.
4. Understood how composite primary keys are used to model transactional and junction tables such as **vendor_inventory** and **vendor_booth_assignments**.
5. Practised writing foundational SQL statements, including **SELECT**, **WHERE**, **ORDER BY**, and **DISTINCT**.
6. Learned to summarise data using aggregate functions such as **SUM()**, **AVG()**, and **COUNT()** in combination with **GROUP BY** and **HAVING**.
7. Gained confidence combining data across multiple tables using **INNER JOIN**, **LEFT JOIN**, and multi-table joins.
8. Learned to write and interpret subqueries, **CASE** expressions, and date/time functions for real-world reporting needs.
9. Understood how window functions such as **RANK()** solve ranking problems that would otherwise require complex procedural code.
10. Developed the ability to translate a real-world business scenario — a farmer's market — into a structured, query-able database system, and to extract actionable insights from it.

13. Conclusion

The **Farmer's Market Management System** project set out to demonstrate that a real-world business scenario — the day-to-day operations of a farmer's market — can be modelled, structured, and analysed entirely through a well-designed relational database and the SQL language. Across the eleven tables that make up the **farmer** database, this project captured every essential aspect of the market: who the vendors are, what they sell, where they sell it, who buys it, and under what conditions each transaction takes place.

Through the process of designing the schema, applying appropriate constraints, and writing progressively more advanced SQL queries, this project reinforced that SQL is not merely a tool for storing data, but a powerful language for asking and answering meaningful business questions. From simple **SELECT** statements to multi-table joins, subqueries, **CASE** expressions, and window functions, each concept built upon the last to demonstrate how raw transactional records can be transformed into actionable insight — identifying top-performing vendors, understanding seasonal sales trends, and spotting inventory or attendance patterns that would be invisible in a simple spreadsheet.

Ultimately, this project has provided a solid, practical foundation in relational database design and SQL query development — skills that are directly transferable to real-world data analyst, database administrator, and backend developer roles across virtually every industry. The techniques demonstrated here on a modest, eleven-table farmer's market database scale naturally to far larger and more complex enterprise systems, making this project a meaningful stepping stone toward more advanced database and data engineering work in the future.